

105. Construct Binary Tree from Preorder and Inorder Traversal Medium

Acceptance: 68.2% | [LeetCode](#)

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:

```
**Input:** preorder = [3,9,20,15,7], inorder =  
[9,3,15,20,7]  
**Output:** [3,9,20,null,null,15,7]
```

Example 2:

```
**Input:** preorder = [-1], inorder = [-1]  
**Output:** [-1]
```

Constraints:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` and `inorder` consist of **unique** values.
- Each value of `inorder` also appears in `preorder`.
- `preorder` is **guaranteed** to be the preorder traversal of the tree.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.

Claude Opus Solution

Construct Binary Tree from Preorder and Inorder Traversal

1. Intuition

The key insight comes from understanding what each traversal tells us:

- **Preorder** traversal visits nodes in the order: `Root → Left Subtree → Right Subtree`. This means the **first element is always the root**.
- **Inorder** traversal visits nodes in the order: `Left Subtree → Root → Right Subtree`. This means once we know the root, **everything to its left belongs to the left subtree** and **everything to its right belongs to the right subtree**.

By combining these two facts: 1. Take the first element of `preorder` → that's the **root**. 2. Find that root in `inorder` → it **splits** the `inorder` array into left and right subtrees. 3. **Recurse** on both halves.

Since all values are unique, we can use a **hash map** to find any value's index in `inorder` in $O(1)$ time instead of scanning.

2. Approach

1. Build a hashmap: `value → index` in `inorder` (for $O(1)$ lookups)
2. Maintain a global pointer `'preorder_index'` starting at 0
 - This pointer consumes `preorder` elements in root-first order
3. Define recursive helper(`left`, `right`) operating on `inorder` boundaries:
 - a. If `left > right`: empty range, return `None` (base case)
 - b. `root_value = preorder[preorder_index]`; advance pointer
 - c. Create root node
 - d. Find root's position in `inorder` using the hashmap
 - e. `root.left = helper(left, position - 1)` # build left subtree first
 - f. `root.right = helper(position + 1, right)` # then right subtree
 - g. return root
4. Return `helper(0, len(inorder) - 1)`

Why build the left subtree first? Because `preorder` is `Root → Left → Right`. After consuming the root, the *next* `preorder` elements belong to the left subtree. The recursion naturally consumes them in order as long as we recurse left before right.

3. Python Solution

```
from typing import List, Optional  
  
class TreeNode:  
    def __init__(self, val=0, left=None, right=None):  
        self.val = val  
        self.left = left  
        self.right = right
```

```

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        # Map each value to its index in the inorder array for O(1) lookup.
        # This avoids scanning inorder on every recursive call.
        inorder_index = {value: i for i, value in enumerate(inorder)}

        # Global pointer into the preorder array.
        # It moves forward exactly once per node (in root-first order).
        # Using a list (or nonlocal) so the inner function can mutate it.
        self.pre_idx = 0

    def build(left: int, right: int) -> Optional[TreeNode]:
        # Base case: the inorder range is empty -> no subtree here.
        if left > right:
            return None

        # The current root is the next unconsumed preorder element.
        root_value = preorder[self.pre_idx]
        self.pre_idx += 1          # consume this element

        # Create the root node.
        root = TreeNode(root_value)

        # Locate the root inside the inorder range.
        # Everything left of `mid` is the left subtree,
        # everything right of `mid` is the right subtree.
        mid = inorder_index[root_value]

        # IMPORTANT: build left subtree first.
        # Preorder is Root -> Left -> Right, so the next preorder
        # elements correspond to the left subtree.
        root.left = build(left, mid - 1)
        root.right = build(mid + 1, right)

        return root

    # Build the whole tree over the full inorder range.
    return build(0, len(inorder) - 1)

```

Quick trace (Example 1)

```

preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
inorder_index = {9:0, 3:1, 15:2, 20:3, 7:4}

build(0,4): root=3 (mid=1)
├ left = build(0,0): root=9 (mid=0) → leaf
└ right = build(2,4): root=20 (mid=3)
    ├── left = build(2,2): root=15 → leaf
    └ right = build(4,4): root=7 → leaf

```

4. Complexity Analysis

	Complexity	Explanation
Time	O(n)	Each node is created exactly once. The hashmap gives O(1) lookups, and the preorder pointer advances n times total.
Space	O(n)	The hashmap stores n entries. The recursion stack uses O(h) where h is tree height — O(n) worst case (skewed tree), O(log n) for balanced trees. Output tree itself is O(n).

Why O(n) and not O(n²)? The naive version that searches inorder linearly and slices arrays each call is O(n²) (and creates copies). Using a hashmap for index lookup + a shared preorder pointer + index boundaries (no slicing) brings it down to O(n).

5. Alternative Approaches

A. Iterative with a Stack — $O(n)$ time, $O(n)$ space

Iterate through preorder maintaining a stack. Push nodes as you go down the left spine; when the top of the stack matches the current inorder value, pop and attach the next preorder node as a right child. Avoids recursion but is trickier to reason about.

B. Naive Recursion with Slicing — $O(n^2)$ time, $O(n^2)$ space

```
def buildTree(self, preorder, inorder):
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    mid = inorder.index(preorder[0]) #  $O(n)$  search
    root.left = self.buildTree(preorder[1:mid+1], inorder[:mid]) #  $O(n)$  slice
    root.right = self.buildTree(preorder[mid+1:], inorder[mid+1:])
    return root
```

Clean and intuitive, but `.index()` and slicing each cost $O(n)$, giving $O(n^2)$. Good for small inputs or interviews to show understanding before optimizing.

Key Patterns to Remember

- **Divide & Conquer on traversal arrays:** first/last element identifies the root, the other traversal partitions subtrees.
- **Hashmap for index lookup** to eliminate repeated linear scans.
- **Shared mutable pointer + boundary indices** to avoid expensive array copies.
- This same technique works for the sister problem **106 (Postorder + Inorder)** — there you take the *last* element of postorder as root and process **right subtree before left**.

Codex Solution

1. Intuition

The key traversal properties are:

- **Preorder** visits nodes as:

```
root -> left subtree -> right subtree
```

So the **first unused value** in `preorder` is always the root of the current subtree.

- **Inorder** visits nodes as:

```
left subtree -> root -> right subtree
```

So once we know the root, we can find it in `inorder` and split the current subtree into:

```
[inorder left part] root [inorder right part]
```

Example:

```
preorder = [3, 9, 20, 15, 7]
inorder  = [9, 3, 15, 20, 7]
```

The first preorder value is 3, so 3 is the root.

In inorder:

```
[9] 3 [15, 20, 7]
```

So:

- Left subtree contains [9]
- Right subtree contains [15, 20, 7]

We recursively repeat the same idea.

Because all values are **unique**, each root has exactly one position in `inorder`.

2. Approach

We avoid slicing arrays because slicing creates copies and can make the solution slower.

Instead:

- Keep a pointer `pre_idx` pointing to the next root in `preorder`.
- Use a hash map to find each value's index in `inorder` in $O(1)$.
- Recursively build the tree using an inorder range `[left, right]`.

Pseudocode

```
buildTree(preorder, inorder):
    create hashmap inorder_index:
        value -> index in inorder

    pre_idx = 0

    define build(left, right):
        if left > right:
            return None

        root_value = preorder[pre_idx]
        pre_idx += 1

        root = TreeNode(root_value)

        mid = inorder_index[root_value]

        root.left = build(left, mid - 1)
        root.right = build(mid + 1, right)

        return root

    return build(0, n - 1)
```

Why build left before right?

Because preorder is:

```
root -> left -> right
```

After consuming the root, the next values belong to the left subtree first.

3. Python Solution

```
from typing import List, Optional
import sys

# Definition for a binary tree node.
# LeetCode already provides this class, so you usually do not need to redefine it.
class TreeNode:
    def __init__(self, val: int = 0, left: Optional["TreeNode"] = None, right: Optional["TreeNode"] = None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        """
        Reconstruct a binary tree from preorder and inorder traversal.

        preorder: root -> left -> right
        inorder:  left -> root -> right
        """

        n = len(preorder)

        # For a skewed tree, recursion depth can become O(n).
        # Python's default recursion limit may be too small for n = 3000.
```

```

sys.setrecursionlimit(max(10000, n + 10))

# Map each value to its index in inorder.
# This lets us find the root position in O(1).
inorder_index = {}
for i, value in enumerate(inorder):
    inorder_index[value] = i

# pre_idx always points to the next root in preorder.
pre_idx = 0

def build(left: int, right: int) -> Optional[TreeNode]:
    """
    Build the subtree whose nodes appear in inorder[left : right + 1].
    """

    nonlocal pre_idx

    # Empty inorder range means no subtree.
    if left > right:
        return None

    # The next preorder element is the root of this subtree.
    root_value = preorder[pre_idx]
    pre_idx += 1

    root = TreeNode(root_value)

    # Find root position in inorder.
    # Everything left of this index is in the left subtree.
    # Everything right of this index is in the right subtree.
    mid = inorder_index[root_value]

    # Build left subtree first because preorder visits left subtree
    # immediately after the root.
    root.left = build(left, mid - 1)

    # Then build the right subtree.
    root.right = build(mid + 1, right)

    return root

return build(0, n - 1)

```

4. Complexity Analysis

Let n be the number of nodes.

Time Complexity: $O(n)$

Each node is processed exactly once.

Also, each lookup of a node's position in `inorder` is $O(1)$ because we use a hash map.

So total time is:

$O(n)$

Space Complexity: $O(n)$

We use:

1. `inorder_index` hash map storing n values: $O(n)$
2. Recursion call stack:
3. Best case for balanced tree: $O(\log n)$
4. Worst case for skewed tree: $O(n)$

Therefore total auxiliary space is:

$O(n)$

5. Alternative Approaches

Alternative 1: Recursive Solution With Array Slicing

You can recursively slice `preorder` and `inorder` arrays for each subtree.

It is simpler conceptually, but inefficient:

```
root = preorder[0]
split inorder around root
recursively build left and right using sliced arrays
```

However, slicing takes extra time and memory, so the worst-case complexity can become:

```
Time:  O(n^2)
Space: O(n^2)
```

This is not ideal.

Alternative 2: Recursive Solution Using Explicit Preorder Ranges

Instead of using a global `pre_idx`, you can pass preorder boundaries into the recursive function.

This also achieves:

```
Time:  O(n)
Space: O(n)
```

But the implementation is slightly more verbose because you need to calculate subtree sizes.

Alternative 3: Iterative Stack-Based Construction

There is also an iterative stack solution using preorder and inorder pointers.

This avoids recursion, but it is harder to reason about and less commonly used in interviews for this problem.

The recursive hashmap solution is usually the clearest and most recommended.